

NS-3 Tutorial#1

Tracing of the results

1. Wireshark (packet analyser)
2. Ascii traces
3. Netanim

NetAnim install instructions

```
$ cd ns-allinone-3.30.1/netanim-3.108/
```

```
$ sudo apt install qt4-default qt4-qmake
```

```
$ qmake NetAnim.pro
```

```
$ make
```

```
$ ./NetAnim
```

References

1. <https://www.nsnam.org/docs/release/3.30/tutorial/ns-3-tutorial.pdf>
2. https://www.youtube.com/watch?v=o_2oi1lpKtg
3. <https://www.youtube.com/watch?v=9rkN3FtOkaQ&t=47s>

NetAnim (show the results through simulation)

```
#include "ns3/netanim-module.h"
```

```
After line 68: AnimationInterface anim("first.xml");
```

```
    anim.SetConstantPosition(nodes.Get(0),10.0,10.0);
```

```
    anim.SetConstantPosition(nodes.Get(1),20.0,20.0);
```

PCAP Files (show the results through simulation)

After line 68: `pointToPoint.EnablePcap ("first_node", devices.Get (0));`

Ascii Traces (show the results through simulation)

After line 68:

```
AsciiTraceHelper ascii;
```

```
pointToPoint.EnableAsciiAll(ascii.CreateFileStream("first.tr"));
```

- +: An enqueue operation occurred on the device queue;
- -: A dequeue operation occurred on the device queue;
- d: A packet was dropped, typically because the queue was full;
- r: A packet was received by the net device.

An enqueue operation on the transmit queue

simulation time expressed in seconds

+

2

/NodeList/0/DeviceList/0/\$ns3::PointToPointNetDevice/

TxQueue/Enqueue

ns3::PppHeader (

Point-to-Point Protocol: IP (0x0021))

ns3::Ipv4Header (

tos 0x0 ttl 64 id 0 protocol 17 offset 0 flags [none]

length: 1052 10.1.1.1 > 10.1.1.2)

ns3::UdpHeader (

length: 1032 49153 > 9)

Payload (size=1024)

what kind of device

NS-3 Tutorial#2

Example 3: Third.cc

```
// Default Network Topology
//
//   Wifi 10.1.3.0
//
//   *      *      *      AP
//   |      |      |      |
// n5  n6  n7  n0  ----- 10.1.1.0
//                        point-to-point
//                        |
//                        -----
//                        LAN 10.1.2.0
//                        |
//                        n1  n2  n3  n4
```

```

38 using namespace ns3;
39
40 NS_LOG_COMPONENT_DEFINE ("ThirdScriptExample");
41
42 int
43 main (int argc, char *argv[])
44 {
45     bool verbose = true;
46     uint32_t nCsma = 3;
47     uint32_t nWifi = 3;
48     bool tracing = false;
49
50     CommandLine cmd (__FILE__);
51     cmd.AddValue ("nCsma", "Number of \"extra\" CSMA nodes/devices", nCsma);
52     cmd.AddValue ("nWifi", "Number of wifi STA devices", nWifi);
53     cmd.AddValue ("verbose", "Tell echo applications to log if true", verbose);
54     cmd.AddValue ("tracing", "Enable pcap tracing", tracing);
55
56     cmd.Parse (argc,argv);
57
58     // The underlying restriction of 18 is due to the grid position
59     // allocator's configuration; the grid layout will exceed the
60     // bounding box if more than 18 nodes are provided.
61     if (nWifi > 18)
62     {
63         std::cout << "nWifi should be 18 or less; otherwise grid layout exceeds the bounding box" << std::endl;
64         return 1;
65     }
66
67     if (verbose)
68     {
69         LogComponentEnable ("UdpEchoClientApplication", LOG_LEVEL_INFO);
70         LogComponentEnable ("UdpEchoServerApplication", LOG_LEVEL_INFO);
71     }
72

```

```
73 NodeContainer p2pNodes;
74 p2pNodes.Create (2);
75
76 PointToPointHelper pointToPoint;
77 pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
78 pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));
79
80 NetDeviceContainer p2pDevices;
81 p2pDevices = pointToPoint.Install (p2pNodes);
82
83 NodeContainer csmaNodes;
84 csmaNodes.Add (p2pNodes.Get (1));
85 csmaNodes.Create (nCsma);
86
87 CsmaHelper csma;
88 csma.SetChannelAttribute ("DataRate", StringValue ("100Mbps"));
89 csma.SetChannelAttribute ("Delay", TimeValue (NanoSeconds (6560)));
90
91 NetDeviceContainer csmaDevices;
92 csmaDevices = csma.Install (csmaNodes);
93
94 NodeContainer wifiStaNodes;
95 wifiStaNodes.Create (nWifi);
96 NodeContainer wifiApNode = p2pNodes.Get (0);
97
98 YansWifiChannelHelper channel = YansWifiChannelHelper::Default ();
99 YansWifiPhyHelper phy;
100 phy.SetChannel (channel.Create ());
101
102 WifiHelper wifi;
103 wifi.SetRemoteStationManager ("ns3::AarfWifiManager");
104
```

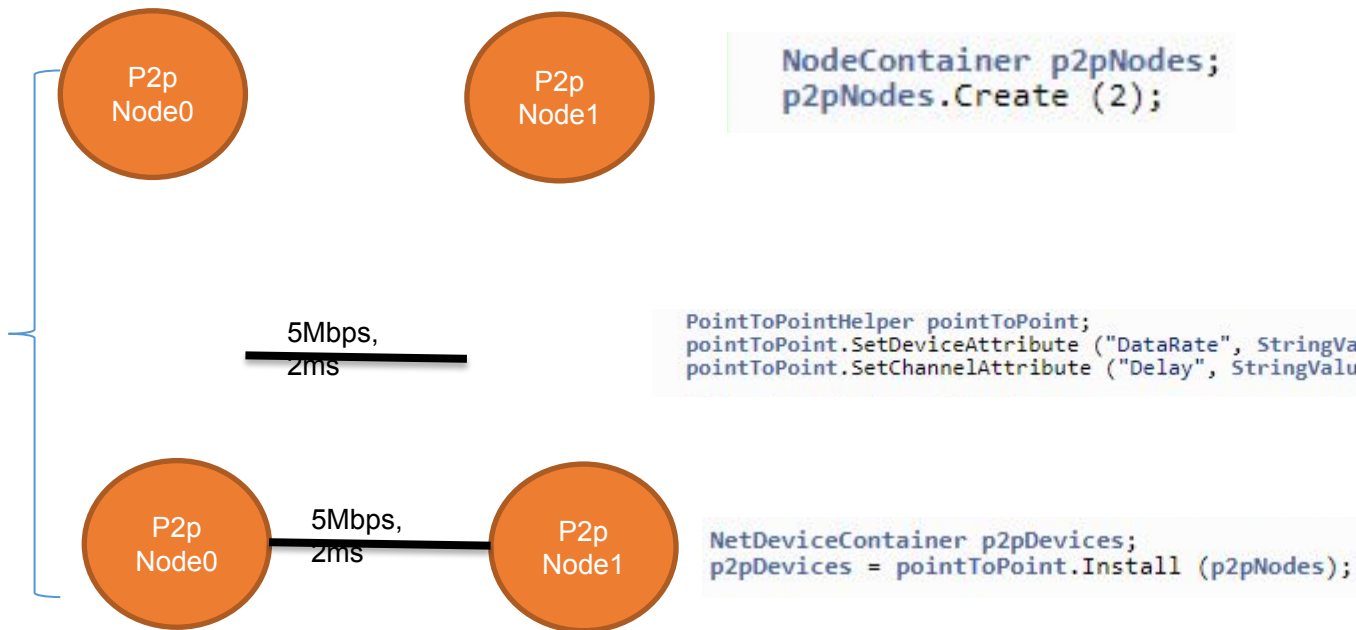
```
105 WifiMacHelper mac;  
106 Ssid ssid = Ssid ("ns-3-ssid");  
107 mac.SetType ("ns3::StaWifiMac",  
108             "Ssid", SsidValue (ssid),  
109             "ActiveProbing", BooleanValue (false));  
110  
111 NetDeviceContainer staDevices;  
112 staDevices = wifi.Install (phy, mac, wifiStaNodes);  
113  
114 mac.SetType ("ns3::ApWifiMac",  
115             "Ssid", SsidValue (ssid));  
116  
117 NetDeviceContainer apDevices;  
118 apDevices = wifi.Install (phy, mac, wifiApNode);  
119  
120 MobilityHelper mobility;  
121  
122 mobility.SetPositionAllocator ("ns3::GridPositionAllocator",  
123                               "MinX", DoubleValue (0.0),  
124                               "MinY", DoubleValue (0.0),  
125                               "DeltaX", DoubleValue (5.0),  
126                               "DeltaY", DoubleValue (10.0),  
127                               "GridWidth", UIntegerValue (3),  
128                               "LayoutType", StringValue ("RowFirst"));  
129  
130 mobility.SetMobilityModel ("ns3::RandomWalk2dMobilityModel",  
131                             "Bounds", RectangleValue (Rectangle (-50, 50, -50, 50)));  
132 mobility.Install (wifiStaNodes);  
133  
134 mobility.SetMobilityModel ("ns3::ConstantPositionMobilityModel");  
135 mobility.Install (wifiApNode);  
136  
137 InternetStackHelper stack;  
138 stack.Install (csmaNodes);  
139 stack.Install (wifiApNode);  
140 stack.Install (wifiStaNodes);  
141
```

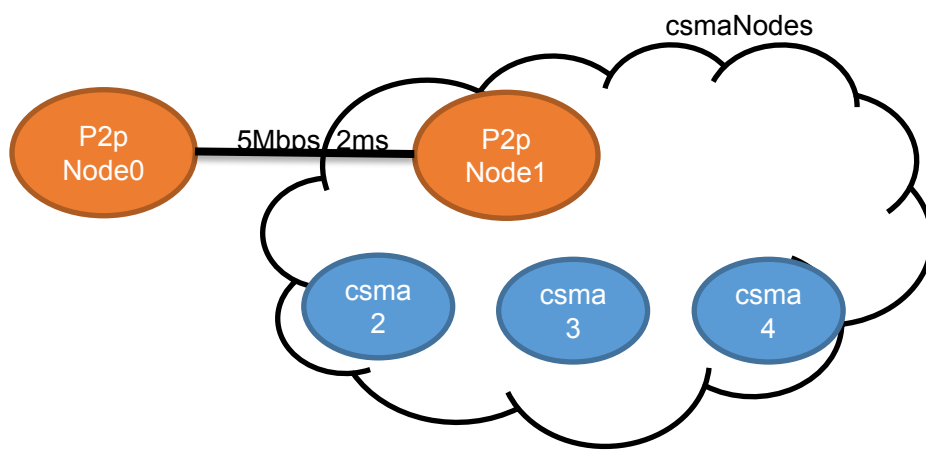
```

142 Ipv4AddressHelper address;
143
144 address.SetBase ("10.1.1.0", "255.255.255.0");
145 Ipv4InterfaceContainer p2pInterfaces;
146 p2pInterfaces = address.Assign (p2pDevices);
147
148 address.SetBase ("10.1.2.0", "255.255.255.0");
149 Ipv4InterfaceContainer csmaInterfaces;
150 csmaInterfaces = address.Assign (csmaDevices);
151
152 address.SetBase ("10.1.3.0", "255.255.255.0");
153 address.Assign (staDevices);
154 address.Assign (apDevices);
155
156 UdpEchoServerHelper echoServer (9);
157
158 ApplicationContainer serverApps = echoServer.Install (csmaNodes.Get (nCsm));
159 serverApps.Start (Seconds (1.0));
160 serverApps.Stop (Seconds (10.0));
161
162 UdpEchoClientHelper echoClient (csmaInterfaces.GetAddress (nCsm), 9);
163 echoClient.SetAttribute ("MaxPackets", UIntegerValue (1));
164 echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
165 echoClient.SetAttribute ("PacketSize", UIntegerValue (1024));
166
167 ApplicationContainer clientApps =
168     echoClient.Install (wifiStaNodes.Get (nWifi - 1));
169 clientApps.Start (Seconds (2.0));
170 clientApps.Stop (Seconds (10.0));
171
172 Ipv4GlobalRoutingHelper::PopulateRoutingTables ();
173
174 Simulator::Stop (Seconds (10.0));
175
176 if (tracing == true)
177 {
178     pointToPoint.EnablePcapAll ("third");
179     phy.EnablePcap ("third", apDevices.Get (0));
180     csma.EnablePcap ("third", csmaDevices.Get (0), true);
181 }
182
183 Simulator::Run ();
184 Simulator::Destroy ();
185 return 0;

```

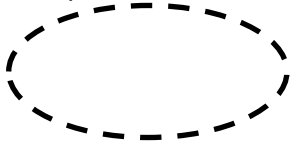

P2P network





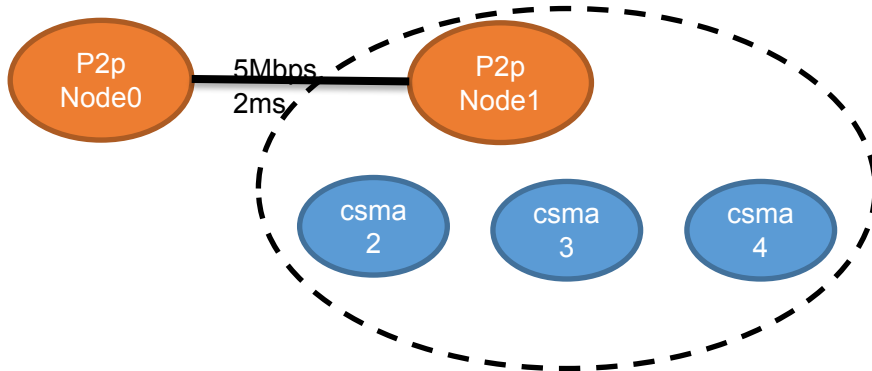
```
NodeContainer csmaNodes;
csmaNodes.Add (p2pNodes.Get (1));
csmaNodes.Create (nCsma);
```

100Mbps,6.56msec

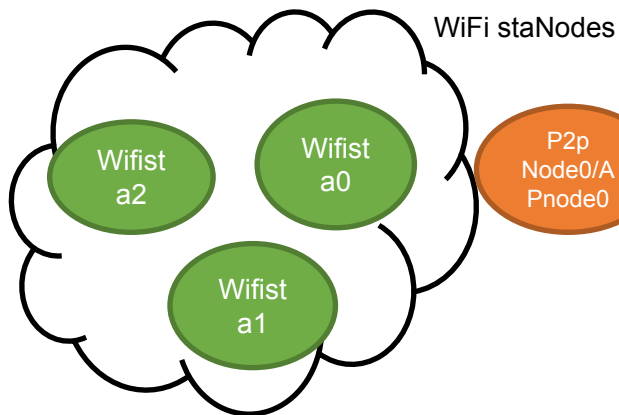


```
CsmaHelper csma;
csma.SetChannelAttribute ("DataRate", StringValue ("100Mbps"));
csma.SetChannelAttribute ("Delay", TimeValue (NanoSeconds (6560)));
```

100Mbps,6.56msec



```
NetDeviceContainer csmaDevices;
csmaDevices = csma.Install (csmaNodes);
```

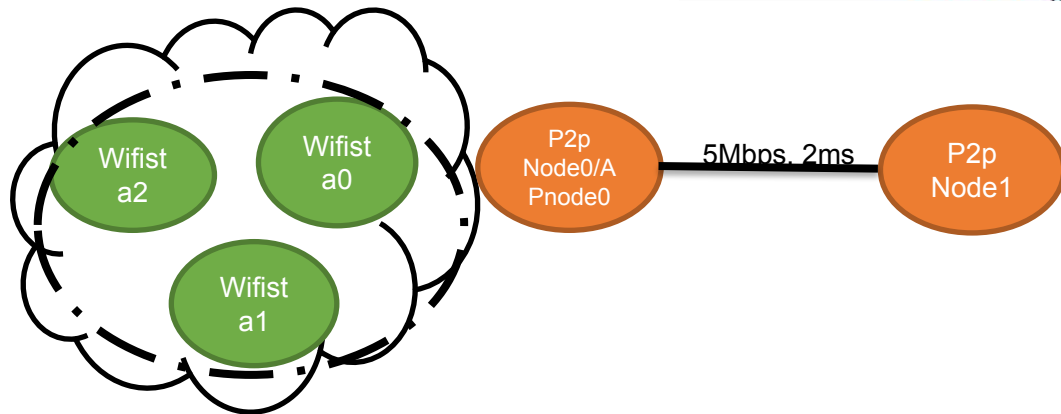
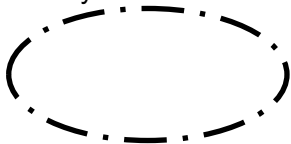
```
NodeContainer wifiStaNodes;
wifiStaNodes.Create (nWifi);
NodeContainer wifiApNode = p2pNodes.Get (0);
```

```
YansWifiChannelHelper channel = YansWifiChannelHelper::Default ();
YansWifiPhyHelper phy;
phy.SetChannel (channel.Create ());
```

```
WifiHelper wifi;
wifi.SetRemoteStationManager ("ns3::AarfWifiManager");
```

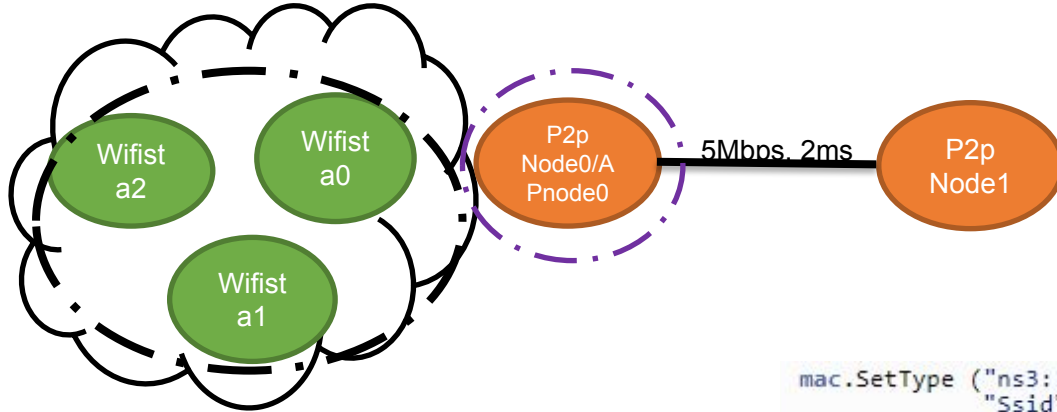
```
WifiMacHelper mac;
Ssid ssid = Ssid ("ns-3-ssid");
mac.SetType ("ns3::StaWifiMac",
            "Ssid", SsidValue (ssid),
            "ActiveProbing", BooleanValue (false));
```

WiFi Physical+Sta mac

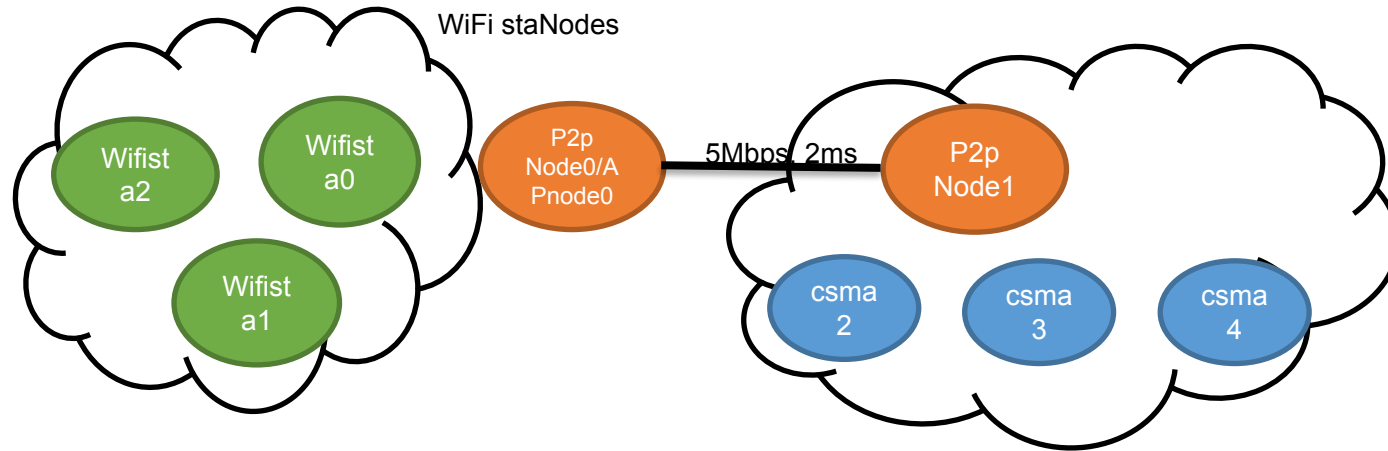


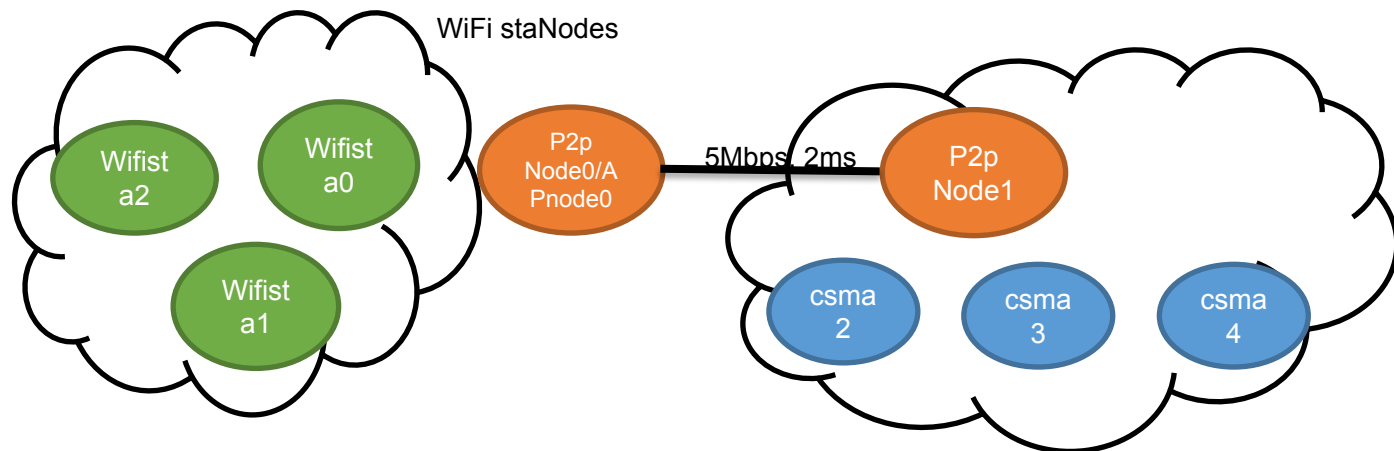
```
NetDeviceContainer staDevices;
staDevices = wifi.Install (phy, mac, wifiStaNodes);
```

WiFi Physical+AP mac



```
mac.SetType ("ns3::ApWifiMac",  
            "Ssid", SsidValue (ssid));  
  
NetDeviceContainer apDevices;  
apDevices = wifi.Install (phy, mac, wifiApNode);
```





```
InternetStackHelper stack;
stack.Install (csmaNodes);
stack.Install (wifiApNode);
stack.Install (wifiStaNodes);

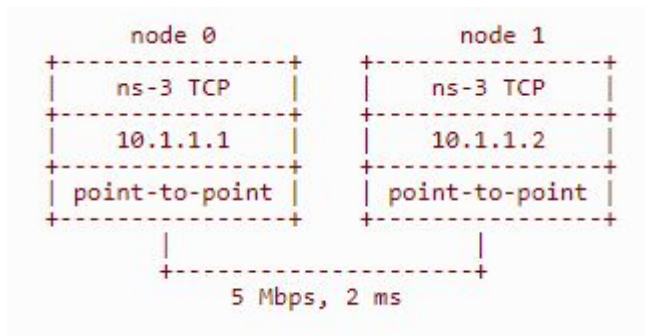
Ipv4AddressHelper address;

address.SetBase ("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer p2pInterfaces;
p2pInterfaces = address.Assign (p2pDevices);

address.SetBase ("10.1.2.0", "255.255.255.0");
Ipv4InterfaceContainer csmaInterfaces;
csmaInterfaces = address.Assign (csmaDevices);

address.SetBase ("10.1.3.0", "255.255.255.0");
address.Assign (staDevices);
address.Assign (apDevices);
```

Example 4: Fifth.cc



```
class MyApp : public Application
{
public:
```

The declaration of the MyApp Application that we put together to allow the Socket to be created at configuration time.

```
    MyApp ();
    virtual ~MyApp();
```

```
    void Setup (Ptr<Socket> socket, Address address, uint32_t packetSize, uint32_t nPackets, DataRate dataRate);
```

```
private:
```

```
    virtual void StartApplication (void);
    virtual void StopApplication (void);
```

```
    void ScheduleTx (void);
    void SendPacket (void);
```

```
    Ptr<Socket>      m_socket;
    Address           m_peer;
    uint32_t          m_packetSize;
    uint32_t          m_nPackets;
    DataRate          m_dataRate;
    EventId           m_sendEvent;
    bool              m_running;
    uint32_t          m_packetsSent;
};
```

```
MyApp::MyApp ()
: m_socket (0),
  m_peer (),
  m_packetSize (0),
  m_nPackets (0),
  m_dataRate (0),
  m_sendEvent (),
  m_running (false),
  m_packetsSent (0)
{
}
```

```

100 MyApp::~MyApp()
101 {
102     m_socket = 0;
103 }
104
105 void
106 MyApp::Setup (Ptr<Socket> socket, Address address, uint32_t packetSize, uint32_t nPackets, DataRate dataRate)
107 {
108     m_socket = socket;
109     m_peer = address;
110     m_packetSize = packetSize;
111     m_nPackets = nPackets;
112     m_dataRate = dataRate;
113 }
114
115 void
116 MyApp::StartApplication (void)
117 {
118     m_running = true;
119     m_packetsSent = 0;
120     m_socket->Bind ();
121     m_socket->Connect (m_peer);
122     SendPacket ();
123 }
124
125 void
126 MyApp::StopApplication (void)
127 {
128     m_running = false;
129
130     if (m_sendEvent.IsRunning ())
131     {
132         Simulator::Cancel (m_sendEvent);
133     }
134
135     if (m_socket)
136     {
137         m_socket->Close ();
138     }
139 }

```

```

141 void
142 MyApp::SendPacket (void)
143 {
144     Ptr<Packet> packet = Create<Packet> (m_packetSize);
145     m_socket->Send (packet);
146
147     if (++m_packetsSent < m_nPackets)
148     {
149         ScheduleTx ();
150     }
151 }
152
153 void
154 MyApp::ScheduleTx (void)
155 {
156     if (m_running)
157     {
158         Time tNext (Seconds (m_packetSize * 8 / static_cast<double> (m_dataRate.GetBitRate ())));
159         m_sendEvent = Simulator::Schedule (tNext, &MyApp::SendPacket, this);
160     }
161 }
162
163 static void
164 CwndChange (uint32_t oldCwnd, uint32_t newCwnd)
165 {
166     NS_LOG_UNCOND (Simulator::Now ().GetSeconds () << "\t" << newCwnd);
167 }
168
169 static void
170 RxDrop (Ptr<const Packet> p)
171 {
172     NS_LOG_UNCOND ("RxDrop at " << Simulator::Now ().GetSeconds ());
173 }
174

```



```

175 int
176 main (int argc, char *argv[])
177 {
178     CommandLine cmd (__FILE__);
179     cmd.Parse (argc, argv);
180
181     NodeContainer nodes;
182     nodes.Create (2);
183
184     PointToPointHelper pointToPoint;
185     pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
186     pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));
187
188     NetDeviceContainer devices;
189     devices = pointToPoint.Install (nodes);
190
191     Ptr<RateErrorModel> em = CreateObject<RateErrorModel> ();
192     em->SetAttribute ("ErrorRate", DoubleValue (0.00001));
193     devices.Get (1)->SetAttribute ("ReceiveErrorModel", PointerValue (em));
194
195     InternetStackHelper stack;
196     stack.Install (nodes);
197
198     Ipv4AddressHelper address;
199     address.SetBase ("10.1.1.0", "255.255.255.252");
200     Ipv4InterfaceContainer interfaces = address.Assign (devices);
201
202     uint16_t sinkPort = 8080;
203     Address sinkAddress (InetSocketAddress (interfaces.GetAddress (1), sinkPort));
204     PacketSinkHelper packetSinkHelper ("ns3::TcpSocketFactory", InetSocketAddress (Ipv4Address::GetAny (), sinkPort));
205     ApplicationContainer sinkApps = packetSinkHelper.Install (nodes.Get (1));
206     sinkApps.Start (Seconds (0.));
207     sinkApps.Stop (Seconds (20.));
208 }

```

ns-3 provides ErrorModel objects which can be attached to Channels. We are using the RateErrorModel which allows us to introduce errors into a Channel at a given rate.

Since we are using TCP, we need something on the destination Node to receive TCP connections and data. The PacketSink Application is commonly used in ns-3 for that purpose

This code instantiates a PacketSinkHelper and tells it to create sockets using the class ns3::TcpSocketFactory

```

163 static void
164 CwndChange (uint32_t oldCwnd, uint32_t newCwnd)
165 {
166     NS_LOG_UNCOND (Simulator::Now ().GetSeconds () << "\t" << newCwnd);
167 }
168

```

calls the static member function `Socket::CreateSocket` and provides a Node and an explicit `TypeId` for the object factory used to create the socket

```

209 Ptr<Socket> ns3TcpSocket = Socket::CreateSocket (nodes.Get (0), TcpSocketFactory::GetTypeId ());
210 ns3TcpSocket->TraceConnectWithoutContext ("CongestionWindow", MakeCallback (&CwndChange));
211
212 Ptr<MyApp> app = CreateObject<MyApp> ();
213 app->Setup (ns3TcpSocket, sinkAddress, 1040, 1000, DataRate ("1Mbps"));
214 nodes.Get (0)->AddApplication (app);
215 app->SetStartTime (Seconds (1.));
216 app->SetStopTime (Seconds (20.));
217
218 devices.Get (1)->TraceConnectWithoutContext ("PhyRxDrop", MakeCallback (&RxDrop));
219
220 Simulator::Stop (Seconds (20));
221 Simulator::Run ();
222 Simulator::Destroy ();
223
224 return 0;
225 }
226

```

Once the `TcpSocket` is created and attached to the Node, we can use `TraceConnectWithoutContext` to connect the `CongestionWindow` trace source to our trace sink.

just stop processing events at 20 seconds into the simulation.

```

105 void
106 MyApp::Setup (Ptr<Socket> socket, Address address, uint32_t packetSize, uint32_t nPackets, DataRate dataRate)
107 {
108     m_socket = socket;
109     m_peer = address;
110     m_packetSize = packetSize;
111     m_nPackets = nPackets;
112     m_dataRate = dataRate;
113 }

```

```

169 static void
170 RxDrop (Ptr<const Packet> p)
171 {
172     NS_LOG_UNCOND ("RxDrop at " << Simulator::Now ());
173 }
174

```

Run

- `$ ./ns3 run examples/tutorial/third.cc -- 3 3 1 1`

Reference

- <https://www.nsnam.org/docs/release/3.42/tutorial/ns-3-tutorial.pdf>
- <https://www.youtube.com/watch?v=KVcZbrMNxvw>
- <https://www.youtube.com/watch?v=9rkN3FtOkaQ>